Graph Intermediate Representation for Dynamic Graph Computation

Abstract

A clear and well-documented LaTeX document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the "acmart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

ACM Reference Format:

. 2018. Graph Intermediate Representation for Dynamic Graph Computation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Graphs are fundamental in modeling relationships and interactions across diverse real-world domains, including social networks, transportation infrastructure, and complex dependency graphs such as AI supply chains and recommendation engines. Directed graphs, in particular, are critical for capturing asymmetric dependencies between entities. Over recent years, graph analytical techniques have undergone rapid development, enabling powerful insights through algorithms like Breadth-First Search (BFS), Single-Source Shortest Path (SSSP), All-Pairs Shortest Paths (APSP), and Betweenness Centrality (BC).

A graph's fundamental structural invariants, such as its connected components, articulation points, or bridges, are pivotal to the efficient execution of many graph algorithms. Optimizations that quickly identify and utilize these structural figures are essential for performance. Therefore, we propose a novel data structure called the Graph Intermediate Representation (GIR) for managing and analyzing dynamic graphs. GIR is explicitly designed to capture and leverage these connected characteristics efficiently, aiming to enable scalable and high-performance execution of common graph algorithms (e.g., BFS, SSSP, APSP, and BC) even under frequent structural updates.

To maximize the benefits of GIR, we also develop complementary optimization strategy for dynamic graph computation: **Scope-Pruned-Update (SPU)**: A strategy designed to reduce the computational scope of updates when dealing with dynamic graph operations (insertions or deletions), thereby minimizing the necessary recomputation.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

https://doi.org/XXXXXXX.XXXXXXX

[YJ: We need to prove a Lemma: Only the insertion from high rank to low rank can form a new SCC, while the insertion happening from low to high or itself will not create any new SCC.] [CQ notes: lemma should be added in section 5.1]

1.1 Context and Challenges in Dynamic Graph Analysis

[YJ: The following is what I have in my proposal. They are dedicated to the AI supply chain graph computation. Please make them more generic, not limited to AI supply chain graph only.]

Modern real-world graphs exhibit three crucial characteristics that introduce significant computational challenges for existing graph systems:

(i) **Frequent Update**: The structure of these graphs is highly dynamic. For instance, our preliminary study shows that one leading AI platform publishes over 3.8K new models and 740 new datasets daily, reflecting a continuous influx of new nodes and edges. This frequency of structural change necessitates constant, low-latency maintenance. (ii) **Ad-hoc Queries**: Insights often arrive continually and require analysts to issue on-the-fly, arbitrary queries. For example, one day a dependency analysis might start from a newly flagged security vulnerability, and the next day, it might trace the impact of a dataset flagged for privacy leakage. (iii) **Large Scale**: Many dynamic systems, such as social graphs or supply chains, continue to grow in size and complexity, potentially reaching billions of nodes and edges.

For these large, frequently updated dynamic graphs, a variety of complex queriesâranging from dependency tracing to connectivity analysisâmust be executed at scale. This context introduces two significant, coupled challenges:

First, **the balance between update efficiency and query responsiveness is broken**. While dynamic graph systems are primarily optimized for efficient graph updates (e.g., insertion/deletion of an edge) [6, 9, 10], the need for fast, repetitive analysis means that graph computation is now at least as critical as the update process itself.

Second, **query patterns are highly arbitrary and non-repetitive**. Many prior dynamic graph techniques assume a fixed query target (e.g., repeatedly updating SSSP from the same source node) [4, 18] to maximize the reuse of cached results. Because modern analytical queries are arbitrary (e.g., a new source node is chosen for every security incident trace), previously cached computations offer little benefit. Consequently, *real-world systems often fall back to recomputing graph analytics from scratch after every update* [7, 25]. This strategy quickly becomes computationally prohibitive as graphs grow and query rates increase.

1.2 Our Contributions

[YJ: Added by me.]

Motivated by these challenges, this paper proposes a novel, high-performance graph computation framework that shifts the paradigm from focusing solely on update speed to achieving fast, scalable responsiveness for arbitrary queries on dynamic graphs. Our main contributions are threefold:

1. Graph Intermediate Representation (GIR). We propose to design a new abstraction called the *Graph Intermediate Representation (GIR)*. Instead of caching results for specific queries, ***our key idea is to proactively maintain structural invariants and summaries*** derived from a small set of foundational graph properties, such as connected components and reachability indexes. This structural persistence provides a unified foundation that significantly accelerates the computation of many diverse ad-hoc graph algorithms.

2. Incremental Maintenance via Scope Pruning. A key challenge is maintaining the consistency and efficiency of the GIR itself in the face of frequent updates (edge insertions/deletions). Recomputing the entire GIR from scratch is computationally prohibitive. We propose a unified method to *incrementally maintain GIR for both primitive and batch updates*. Our primary focus is on the single edge update, as node operations can often be reduced to adjacent edge operations. We introduce the novel Scope-Prune-Update (SPU) strategy, which consists of three steps designed to minimize the affected area within the graph structure, ensuring efficient, targeted maintenance.

3. Optimized Traversal Algorithms. We design and integrate specialized graph algorithms, such as the Rank-Update-Search (RUS) approach, which is optimized to leverage the connectivity information encoded within the GIR during fundamental traversal-based operations (like BFS, SSSP, and BC) to achieve state-of-the-art performance on highly dynamic graphs.

4. Experiment. [YJ: TODO.] We validate the efficiency and scalability of the GIR-based framework against state-of-the-art dynamic graph systems using both real-world and synthetic datasets.

2 Background

Traditional representations of dynamic graphs face challenges in efficiently updating and querying structural properties, especially in directed settings. In this section, we review classical methods of graph representation, summarize recent progress in strongly connected component (SCC) detection, and introduce the block-cut tree (BCT) as a fundamental structure for analyzing connectivity.

2.1 Classical Methods of Graph Representation

Graphs are typically represented using adjacency matrices or adjacency lists. The adjacency matrix provides constant-time edge lookups but requires $O(n^2)$ space, which becomes prohibitive for large and sparse graphs. The adjacency list offers a more space-efficient alternative with $O(n+m)$ storage, where n and m are the numbers of vertices and edges, respectively, and supports efficient traversal operations. For dynamic graphs, however, both representations exhibit limitations. Adjacency matrices are expensive to update under edge insertions and deletions, while adjacency lists may require costly restructuring to maintain efficient queries. Recent research has therefore explored hybrid or hierarchical data structures to balance the trade-off between query performance and update efficiency, especially in large-scale or streaming graph settings.

2.2 WCC Detection

Weakly Connected Components (WCCs) can be efficiently detected by first simplifying the graph using trimming techniques and then applying traversal-based algorithms on the reduced graph. **Trimming** aims to quickly identify trivial SCCs to reduce graph size before WCC detection. **Trim-1** identifies size-1 SCCs: any vertex with zero in-degree or zero out-degree forms a single-vertex SCC. **Trim-2** identifies size-2 SCCs: pairs of vertices connected to each other and satisfying the "single direction rule," which ensures they cannot belong to larger SCCs. These trimming steps iteratively reduce the graph, simplifying subsequent analysis.

After trimming, the **Forward-Backward (FW-BW) BFS algorithm** is applied to detect the remaining SCCs, which form the basis for WCC identification. Starting from a pivot vertex v, FW-BW first constructs the forward set $FW(v)$ of vertices reachable via out-edges and then the backward set $BW(v)$ via in-edges. The intersection $FW(v) \cap BW(v)$ defines an SCC. State-of-the-art methods often treat large and small SCCs differently: large SCCs are directly detected via FW-BW BFS, while small SCCs may be preprocessed using trimming, followed by FW-BW BFS or other lightweight algorithms. Once SCCs are identified, grouping them yields the WCCs of the graph.

2.3 SCC Detection

Efficient detection of strongly connected components (SCCs) in large graphs is a fundamental problem in graph analytics. Traditional approaches often rely on traversal strategies based on breadth-first search (BFS), which face a trade-off between synchronization overhead and workload balance. Specifically, synchronous (Sync) traversal provides balanced workloads but incurs costly level synchronizations, while asynchronous (Async) traversal avoids these synchronizations but easily suffers from workload imbalance.

To address this gap, the ISPAN framework introduces a new synchronization paradigm called *relaxed synchronization* (Rsync). Rsync reduces synchronization overhead while preserving balanced workloads by allowing earlier termination conditions during traversal. Building on Rsync, ISPAN integrates synchronous, asynchronous, and relaxed synchronous strategies into a fast spanning tree construction method, which significantly accelerates SCC detection. The framework further enhances performance through optimized trimming techniques, including an extended trim-3 strategy, and scales efficiently across both shared-memory and distributed environments.

ISPAN demonstrates substantial improvements over existing DFS- and BFS-based methods, achieving order-of-magnitude speedups on real-world and large-scale synthetic graphs, and proving its effectiveness for billion-vertex graph datasets.

2.4 Block-Cut Tree

The block-cut tree (BCT) is a classical data structure used to represent the biconnected components (blocks) and articulation points (cut vertices) of a graph. A block is a maximal subgraph that does not contain any articulation point, while a cut vertex is a node

whose removal increases the number of connected components in the graph. The BCT is constructed by treating each block and cut vertex as nodes in a bipartite tree: an edge connects a block node to a cut-vertex node if the vertex belongs to that block.

This representation offers several advantages. First, it captures the hierarchical decomposition of a graph into fundamental connectivity units, which simplifies both theoretical analysis and algorithmic design. Second, many graph problems, such as articulation point detection, resilience analysis, and network reliability, can be solved more efficiently by leveraging the structure of the BCT. Finally, since the BCT is a tree, it supports linear-time algorithms and provides a compact way to maintain critical connectivity information for complex graphs.

[YJ: Add more if needed.]

3 Overview

We define GIR as a modular abstraction of a dynamic directed graph, enabling algorithmic reuse and fast structural access. Specifically, it organizes graph data based on its connectivity characteristics:

1. Weakly Connected Components (WCC) Approach: A disjoint set (union-find) structure is used to maintain WCCs. 2. Strongly Connected Components (SCC) Approach: Each SCC is contracted into a supernode, forming a condensation graph, which is a Directed Acyclic Graph (DAG). 3. Biconnected Components (BCC) Although more commonly used in undirected graphs, for edge-resilience analysis we extend BCC-like decomposition to understand critical edges and fallback paths in directed contexts.

[YJ: TODO: Add the lemmas] Lemma 1: Given a directed graph, separate graph into different SCCs, then all BCCs are already within every SCCs(directed graph)

Lemma 2: when insertion happens in one DAG, only nodes with topological order from high-level to low-level nodes will be affected

Lemma 3: when check cycle or do path detection in DAG updation, we can prune graph only within the vertice's topological order between source-level and target-level

4 A Novel Hierarchical Graph Intermediate Representation (GIR)

[YJ: Added by me.]

GIR design. Specifically, we propose a hierarchical design for GIR that captures both global and local structure through fundamental graph decompositions, including weakly connected component (WCC), strongly connected components (SCCs) and biconnected components (BiCCs) [12, 15, 17, 23, 24]. They offer structural modularity and efficient computability. Figure **??** overviews the GIR for an example graph. First, we compute WCCs and SCCs, where an SCC is a subgraph of a WCC. Each SCC is then represented as a supernode, yielding a directed acyclic graph (DAG) that reflects the global dependency flow. Within an SCC, each node has at least one directed path to all the others. Due to that, each SCC represents a locally cyclic and tightly coupled set of nodes, which can enable fast downstream computation, e.g., coarse-grained traversal, and reachability estimation. For the example graph in Figure **??**, each shaded area represents an SCC.

Then, we focus on the internal structure of each SCC. We perform BiCC decomposition to detect articulation points (APs), whose removal would disconnect the graph, and BiCCs are maximal subgraphs where the removal of any single node does not disconnect the subgraph. In particular, we compute APs and BiCCs within each SCC by temporarily treating the edges as undirected. This is motivated by the structural property that, under this transformation, an AP in the SCC serves as a critical connector between adjacent BiCCs. Specifically, (i) removing the AP disconnects the SCC into multiple subregions, thus breaking its strong connectivity; and (ii) all the nodes in one BiCC must pass this AP to reach any nodes in the adjacent BiCC. For the example in Figure **??**, to find a path from 2 to 11, it has to pass two APs 3 and 10. After that, we represent each SCC as a *block-cut tree (BC tree)*, which is a tree structure with only two types of nodes, i.e., a block node represents a BiCC, the other node represents an AP. As shown in Figure **??**, a supernode D_1 in the DAG is represented as the BC tree with four nodes, i.e., 3 and 10 are APs, and B_1, B_2 are block nodes. Further, each block node is represented as a subgraph of the original. For dynamic graph formats, we plan to integrate with existing efforts, e.g., PMA [3], DCSR [19], Terrace [22], and DGAP [14].

Benefits of GIR decomposition. The WCC, SCC, and BiCC layers give two key advantages. (i) **Query-agnostic structure.** They summarize global connectivity and resilience once, then serve many tasks, such as, reachability, traversal, path finding [11, 21, 23]. For example, if two nodes belong to the same SCC we can answer reachability immediately, and a path search only needs to explore the BiCCs that contain the endpoints. (ii) **Parallel efficiency**. These decompositions map cleanly to multi-threaded and GPU architectures. Our previous work Aquila [15] and iSpan [16, 17] already compute WCC, SCC, and BiCC on billion edge graphs at high throughput, making them a practical backbone for AI supply chain graph.

Generability of GIR. Although the current GIR uses only WCC, SCC, and BiCC, the abstraction is foundational and open-ended: new structural algorithms (e.g., dominator trees, reachability indices) can be plugged in whenever they help. This makes the framework useful for any large, dynamic graph that needs to process frequent update and ad-hoc queries.

5 GIR Maintenance Under Dynamic Update

[YJ: TODO: How do we dynamically update the GIR? Please write as much details as possible]

5.1 Scope-Prune-Update for Primitive Updates

We propose a unified scope-prune-update (SPU) strategy for primitive updates, mainly focusing on *single-edge operations* (insertions or deletions). Node insertions without any incident edges do not impact the connectivity structure and can be safely ignored in GIR updates. Similarly, node deletions can be treated as bulk deletions of all adjacent edges. Hence, the primary focus is on edge-level operations, including both edge insertions and deletions. With that, we propose a novel scope-prune-update (SPU) strategy including three major steps.

(i) Scope. The first step identifies the minimal subgraph potentially affected by the update. For example, adding an edge that connects two distinct SCCs may cause them to merge, while deleting a bridge in a BiCC could fragment it into smaller components.

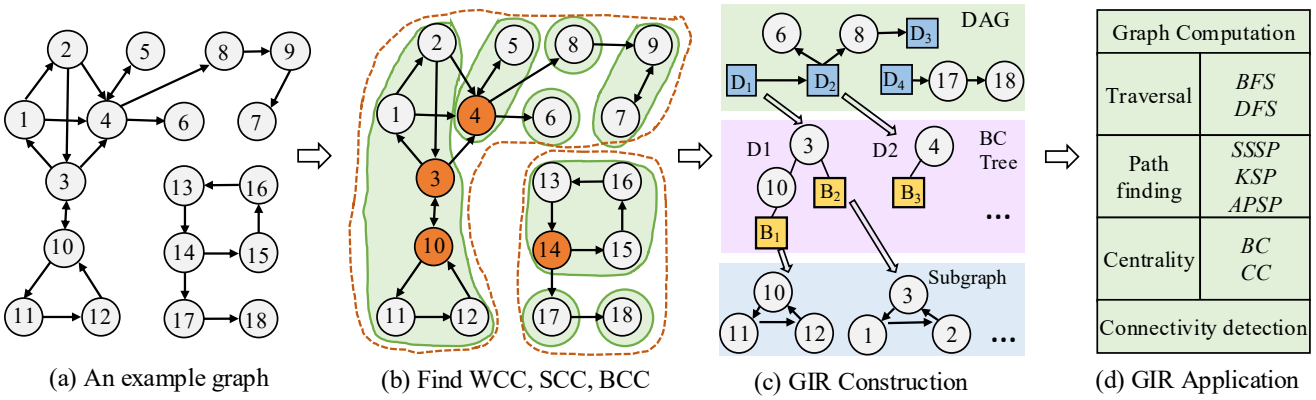


Figure 1: Graph intermediate representation work

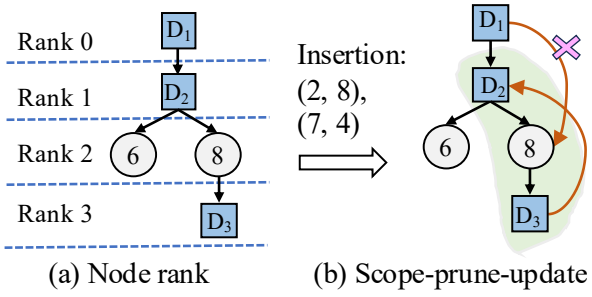


Figure 2: Scope-Prune-Update. **[YJ: For (b), we should consider using a different name than "topological order".]**

We leverage compact metadata such as component identifiers, topological ordering, and articulation points to quickly localize the impact region. This is inspired by dynamic graph systems such as DynoGraph [13] and incremental connectivity frameworks [1], but adapted to structural abstractions in GIR rather than algorithm-specific outputs. [YJ: Not concrete enough, might add an example. Similar for the next two steps.]

To handle updates within DAGs, we maintain rank information for each node. This rank, similar to a topological order, starts from rank 0 and is assigned throughout the DAG.

We observe that if an update occurs from a node with a lower rank to one with a higher rank, it does not affect the DAG's connectivity. Therefore, such updates can be safely ignored in terms of structure as they are connectivity-unaffected updates.

However, if an update goes from a higher-ranked node to a lower-ranked one, it may introduce a cycle. In this case, we first identify the affected scope, which includes only the nodes with ranks between the lower and higher ranks involved. We then perform cycle detection within this range. If a cycle is detected, we apply a minimal update to adjust the DAG accordingly.

(ii) Prune. Once the affected zone is identified, we prune irrelevant vertices and edges using structural heuristics. These include exploiting topological constraints (e.g., descendants in a DAG), edge directionality, and local articulation structures. For instance, if an inserted edge does not complete a cycle, the SCCs may remain unchanged. Such pruning minimizes the recomputation footprint, allowing selective recomputation only where structural transitions are possible.

Only deletions on bridge edges are relevant. Such deletions split the DAG and remove the corresponding BCC: `{(source, target)}`.

Type	Scope	Insertion	Deletion
WCC	Intra	NA	Update
	Inter	O(1) merge	———
SCC	Intra	NA	Update
	Inter	DAG update	NA
BCC	Intra	NA	Update
	Inter	Update	Update

Table 1: Comparison of Insertion and Deletion Impact on Different Connectivity Types

(iii) Update. Finally, we perform fast, local recomputation within the pruned region. For SCCs, we use incremental variants of Tarjan's algorithm or union-find with path compression to efficiently update component labels. For BiCCs, we apply DFS-based articulation point detection limited to the local subgraph. These updates ensure that the GIR remains correct while avoiding full reprocessing of the entire graph. By iteratively applying the SPU strategy, we achieve scalable and responsive GIR maintenance under continuous edge updates. [YJ: This needs to be parallel.]

During dynamic updates of GIR (Graph Irregular Redundancy), it is crucial to maintain the structural information of WCC (Weakly Connected Components), SCC (Strongly Connected Components), and BCC (Biconnected Components). The primary goal is to **minimize the scope of updates** during dynamic operations, **avoid global recalculation**, and thereby **improve overall update efficiency**.

(iii) WCC Maintenance Updating WCCs is relatively straightforward:

– **Internal edge insertion:** Does not affect WCC partitioning.

- **Internal edge deletion:** May cause the current WCC to split, requiring a partial recalculation of the affected component.

(iii) SCC Maintenance The impact scope is efficiently determined by maintaining the **topological level**:

- **Edge insertion within an SCC:** No effect on SCC; no update required.
- **Edge deletion within an SCC:** Check whether the in-degree or out-degree of a vertex becomes zero.
  - If a vertex reaches degree zero, remove it from the current SCC.
- **Edge insertion across SCCs:**
  - If the insertion forms a reverse edge to a parent or ancestor node in the topological order, a new cycle may be created, triggering SCC merging.
  - Otherwise, the SCC structure remains unchanged.

(iii) BCC Maintenance Dynamic updates to BCCs are more complex, requiring careful handling of **cut vertices** and **local structural changes**:

- **Internal edge insertion:** Does not affect existing BCCs; no update required.
- **Internal edge deletion:**
  1. Inspect the affected BCC.
  2. Remove vertices with degree ≤ 1 from the current BCC.
  3. Update the degrees of neighboring vertices; if any neighbor also has degree ≤ 1, recursively remove it until no eligible vertices remain.
- **Edge insertion across BCCs:**
  - Use BFS and topological sorting to detect whether a loop forms between cut vertices.
    - If some nodes remain unvisited, a cycle exists; merge the relevant BCCs.
    - If all nodes are visited, no cycle exists; the original BCCs remain unchanged.
- **Edge deletion across BCCs:**
  - Typically does not affect the current BCC.
  - If the deleted edge belongs to another BCC, follow that BCC's internal deletion procedure.

5.2 Filter-Merge-Update for Batch Updates

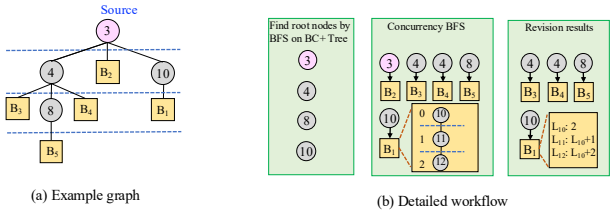
[YJ: Please read the following and think about plotting a figure.]

In practice, the graph changes are likely to arrive in batches, e.g., a new model repository may simultaneously add multiple edges indicating dataset and model dependencies. Processing such updates naively, one edge at a time, is both inefficient and potentially incorrect due to interdependencies among updates. To address this, we propose a filter-merge-update (FMU) pipeline for batch updates.

(i) Filter. We pre-process the update batch to remove redundant or canceling operations. For example, if an edge is added and then deleted in the same batch, it has no net effect. We also discard operations on isolated nodes or those that do not affect the existing GIR (e.g., inserting an edge between already-connected nodes within the same biconnected component).

(ii) Merge. After filtering, we canonicalize the update sequence by grouping together operations on related subgraphs and determining an optimal application order. This is critical because update order can affect both correctness and performance. For instance, deleting an edge and then re-inserting it should not cause the same recomputation steps twice. Merging also allows batch operations to be coalesced into bulk modifications, enabling shared recomputation for overlapping updates. [YJ: Could add more details.]

(iii) Update. We reuse the scope-prune-update strategy proposed for primitive updates, but now applied to merged regions. For example, a group of edges that form a cycle when added together may merge multiple SCCs. Our system detects such structural shifts and updates the GIR holistically using global-to-local propagation.



(a) Example graph (b) Detailed workflow

Figure 3: concurrent BFS by using the figure of BCT

6 A Hierarchical Concurrent Computation Framework

[YJ: Maybe we should switch 6.1 and 6.2]

6.1 A Multi-Step Concurrent Computation Method for Graph Traversal

[YJ: Also, this is from my proposal writing.]

We have built a hierarchical GIR. With that, we *aim to design a scalable computation framework.*

This task proposes to design a *multi-step concurrent computation method* atop our GIR. In the following, we first apply our strategy on breadth-first search (BFS), which is an important graph traversal algorithm. In AI supply chain analytics, traversal plays a pivotal role in answering critical queries, such as, *dependency resolution* (e.g., "what dataset does this model depend on?"), *backward provenance tracing* (e.g., "which models were influenced by a compromised dataset?"), and *forward impact propagation* (e.g., "how widely can a vulnerability in a shared dependency propagate?").

Mutli-step computation on GIR. *Step 1: DAG-level pruning.* We begin by performing a traversal over the DAG to scope and prune the graph. Because the DAG captures global dependencies across SCCs and is significantly smaller than the full graph, this step efficiently identifies the reachable subgraph. That is, only SCCs reachable from the source node's SCC are retained for further traversal. This early pruning greatly reduces computational overhead and makes BFS work-efficient, especially for bottom-up traversal [2] as the workload contains all the unvisited nodes. The time complexity of this step is $O(|V_{DAG}| + |E_{DAG}|)$.

Step 2: BC tree traversal. For the remaining reachable subgraph, we treat it as a unified BC tree by unifying the retained SCCs. Starting from source, we run another BFS on the BC tree to find root node for each block. The time complexity for tree traversal is $O(|V_T|)$. For the example in Figure **??**(a), the root node of each block is its parent.

Step 3: Concurrent BFS. With the block roots determined, we run multiple concurrent BFSes as each block is treated as an independent task, allowing us to exploit task parallelism, e.g., each BFS per GPU cooperative thread array (CTA). Task parallelism can minimize the communication and synchronization overhead as each task is independent. In addition, within each block, we can still perform data-parallel BFS similar to conventional BFS. To this end, we get the local level relative to the block's root. The time complexity is $O(|V_s| + |E_s|)$, where V_s and E_s denote the nodes and edges in scope, respectively. For the example in Figure **??**, five BFSes can be concurrently executed.

Step 4: Final level refinement. Finally, we perform refinement to get the correct BFS levels. Specifically, for each node, its final BFS level is refined as the sum of root node's absolute level and the relative level. As shown in Figure **??**(c), the nodes in four blocks will be refined. Its time complexity is $O(|V_s|)$.

In summary, *the time complexity is dominated by step 3*, as usually $|V_s| \gg |V_T|$ and $|V_s| \gg |V_{DAG}|$. As we can greatly improve the parallelism of this step with concurrent BFS, compared to conventional BFS with data-parallelism only, our method has the potential to dramatically improve the runtime performance. In addition, *this task-parallel decomposition (step 3) can also be applied to depth-first search (DFS)*, offering a promising method for parallelizing a traditionally sequential and hard-to-parallelize problem.

6.2 A Hierarchical Computation Framework

[YJ: Added by me.] [CQ notes: If a new SCC is formed, how could we quickly find it? 1. This only affects the vertices having the ranks between the source and target. 2. FW-BW, color propagation. Bi-direction BFS, meet-in-the-middle .]

Following the previous section, this task aims to efficiently compute a suite of fundamental graph algorithms for dynamic graph analysis, including (i) *reachability* for trace vulnerability or data propagation, (ii) *path finding* for identifying fine-tuning chains, and (iii) *centrality* for assessing importance or influence of critical nodes.

Fast unordered traversal (reachability). When traversal order is not required (e.g., reachability), we can aggressively prune irrelevant regions. From the DAG, after running BFS, only the supernodes that contain source or target nodes need to be further traversed, while all other supernodes can be safely pruned. Within each selected supernode, which is internally structured as a BC tree, the same pruning principle applies, i.e., only the blocks containing source or target nodes need to be traversed, while others are skipped.

Path finding algorithms aim to identify paths on AI supply chain under different constraints, such as, single-source shortest path (SSSP), k-shortest path (KSP), and all-pairs shortest path. Similar to BFS, at the top level, we find possible paths over the DAG to determine which SCCs may participate in the global path using fast DAG traversal and prune the irrelevant ones. Then, we analyze the BC tree of the selected supernodes to identify the root node of relevant BiCCs. After that, we perform both task- and data-parallelism of exact path finding algorithms (e.g., Δ-stepping for SSSP [20], our previous work PeeK for KSP [8]) on each subgraph. Finally, we refine the correct distances and paths based on relative results.

Centrality algorithms such as betweenness centrality and closeness centrality are critical for assessing the relative importance of node/edge in AI supply chain, e.g., identifying influential models, pivotal datasets, or common software dependencies [5, 26]. At their core is computing the shortest paths between node pairs. Therefore, we can extend beyond the previously established method for path finding.

7 Experiments

7.1 Experimental Setup

Our experiments were conducted on a server equipped with a dual-socket (2-way) configuration featuring the AMD EPYC 9334 32-Core Processor. The system utilizes 2 physical CPU chips (Sockets). Each physical chip contains 32 physical cores. With Simultaneous Multithreading (SMT) enabled, the system provides a total of 128 logical processors (threads) for application use.

We conduct all experiments on a system configured with GCC and OpenMP. The environment ensures reproducibility and efficient parallel computation.

7.2 Basic Graph Algorithm Experiments

To evaluate the performance of GIR on standard graph tasks, we perform experiments on the following algorithms:

- Breadth-First Search (BFS)
- Single-Source Shortest Paths (SSSP)
- Betweenness Centrality (BC)

7.3 Datasets

We use several large-scale, symmetrized graph datasets. Table 2 summarizes the number of vertices, edges, and average degree.

Table 2: A list of (symmetrized) graph datasets, including the number of vertices, edges, and average degree.

Dataset	Vertices	Edges	Avg. Degree
LiveJournal	4,847,571	85,702,474	17.8
Orkut	3,072,627	234,370,166	76.2
rMAT	8,388,608	563,816,288	60.4
Protein	8,745,543	1,309,240,502	149.7
Twitter	61,578,415	2,405,026,092	39.1

7.4 System Descriptions

We compare GIR with the following graph representation systems:

- Terrace [**?**]
- CuckooGraph [**?**]
- Aspan
- Spruce [**?**]

7.5 Update Throughput

7.5.1 Setup We measure update throughput for edge insertions, deletions, and queries on all datasets. The setup includes varying batch sizes and concurrency levels to evaluate system scalability.

7.5.2 Results

Edge Insertion GIR consistently achieves higher insertion throughput compared to Terrace, CuckooGraph, Aspan, and Spruce.

Edge Deletion Edge deletion performance shows similar trends, with GIR outperforming others due to its optimized memory management.

Edge Query Query throughput remains high across all datasets, demonstrating GIRâs efficiency in maintaining up-to-date graph structures.

7.5.3 Discussion Overall, GIR provides better update performance under dynamic graph changes. Our design achieves a good balance between concurrency and memory efficiency.

7.6 Parameter and Ablation Experiments

We explore the effect of different parameter settings on GIR's performance:

- Varying internal buffer sizes
- Different parallelism strategies (OpenMP vs OpenCilk)
- Effect of selective pruning and optimizations

Ablation studies show that each optimization contributes positively to throughput and memory efficiency. Removing any key optimization leads to a noticeable performance drop.

7.7 Graph Analysis Experiments

We evaluate GIR on standard graph analysis algorithms:

Breadth-First Search (BFS) BFS is executed on all datasets. GIR achieves lower execution time due to its optimized traversal strategy.

Single-Source Shortest Paths (SSSP) SSSP experiments show GIR scales efficiently with graph size, outperforming existing systems in both time and memory usage.

Betweenness Centrality (BC) BC computation demonstrates GIRâs capability in handling large-scale graphs, leveraging internal optimizations for redundant computation elimination.

8 Releted Work

9 Conclusion

Graph Intermediate Representation is

References

[1] Chirag Agarwal, Carsten Binnig, Semih Salihoglu, and Richard Sidle. 2020. Incremental View Maintenance for Graph Queries. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, 1455–1470. doi:10.1145/3318464.3380572

[2] Scott Beamer, Krste Asanovic, and David Patterson. 2012. Direction-optimizing breadth-first search. In *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–10.

[3] Michael A Bender and Haodong Hu. 2007. An adaptive packed-memory array. *ACM Transactions on Database Systems (TODS)* 32, 4 (2007), 26–es.

[4] Aaron Bernstein and Liam Roditty. 2016. Maintaining shortest paths under deletions in weighted directed graphs. *SIAM J. Comput.* 45, 6 (2016), 2089–2116.

[5] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology* 25, 2 (2001), 163–177.

[6] David Ediger, Kaycee Jiang, James Riedy, and David A Bader. 2012. STINGER: High performance data structure for streaming graphs. In *IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–5.

[7] David Eppstein, Zvi Galil, Giuseppe F Italiano, and Amnon Nissenzweig. 2013. Dynamic graph algorithms. In *Handbook of Algorithms and Data Structures*. CRC Press, 585–636.

[8] Wang Feng, Shiyang Chen, Hang Liu, and Yuede Ji. 2023. PeeK: A Prune-Centric Approach for K Shortest Path Computation. In *SC*.

[9] Hongru Gao, Xiaofei Liao, Zhiyuan Shao, Kexin Li, Jiajie Chen, and Hai Jin. 2024. A survey on dynamic graph processing on GPUs: concepts, terminologies and systems. *Frontiers of Computer Science* 18, 4 (2024), 184106.

[10] Oded Green, Robert McColl, David A Bader, Duane Merrill, Ahmet Erdem Sariyuce, George Slota, Umit V Catalyurek, et al. 2021. Hornet: An Efficient Data Structure for Dynamic Graphs on GPUs. In *2021 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–7.

[11] Jonathan L Gross and Jay Yellen. 2005. *Graph theory and its applications*. CRC press.

[12] John E Hopcroft and Robert E Tarjan. 1973. Algorithm 447: Efficient algorithms for graph manipulation. In *Communications of the ACM*, Vol. 16. ACM, 372–378.

[13] Wenguang Huang, Yongkun Zhao, Zheng Xu, Wenguang Han, Chun Jason Xue, and Yun Liu. 2022. DynaGraph: Dynamic Graph Processing with Efficient Snapshot Isolation. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM, 151–164. doi:10.1145/3503221.3508411

[14] Abdullah Al Raqibul Islam and Dong Dai. 2023. DGAP: Efficient dynamic graph analysis on persistent memory. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–13.

[15] Yuede Ji and H. Howie Huang. 2020. Aquila: Adaptive Parallel Computation of Graph Connectivity Queries. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*.

[16] Yuede Ji, Hang Liu, Yang Hu, and H. Howie Huang. 2022. ispan: Parallel identification of strongly connected components with spanning trees. In *ACM Transactions on Parallel Computing*. ACM.

[17] Yuede Ji, Hang Liu, and H. Howie Huang. 2018. iSpan: Parallel Identification of Strongly Connected Components with Spanning Trees. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 731–742.

[18] Arindam Khanda, Sriram Srinivasan, Sanjukta Bhowmick, Boyana Norris, and Sajal K Das. 2021. A parallel algorithm template for updating single-source shortest paths in large-scale dynamic networks. *IEEE Transactions on Parallel and Distributed Systems* 33, 4 (2021), 929–940.

[19] James King, Thomas Gilray, Robert M Kirby, and Matthew Might. 2016. Dynamic sparse-matrix allocation on GPUs. In *International Conference on High Performance Computing*. Springer, 61–80.

[20] Ulrich Meyer and Peter Sanders. 2003. Δ-Stepping: A Parallel Single Source Shortest Path Algorithm. *Journal of Algorithms* 49, 1 (2003), 114–152.

[21] Hiroshi Nagamochi and Toshihide Ibaraki. 2000. Graph algorithms for network connectivity and reliability. *Networks* 36, 2 (2000), 96–103.

[22] Prashant Pandey, Brian Wheatman, Helen Xu, and Aydin Buluc. 2021. Terrace: A Hierarchical Graph Container for Skewed Dynamic Graphs. In *Proceedings of the 2021 International Conference on Management of Data*. 1372–1385.

[23] Robert Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM journal on computing* 1, 2 (1972), 146–160.

[24] Robert E Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM J. Comput.* 1, 2 (1972), 146–160.

[25] Alexander van der Grinten, Markus Brand, and Peter Sanders. 2022. An Evaluation of Data Structures for Fully Dynamic Graphs. In *International Symposium on Experimental Algorithms*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 162–178.

[26] Lei Wang, Fan Yang, Liangji Zhuang, Huimin Cui, Fang Lv, and Xiaobing Feng. 2016. Articulation points guided redundancy elimination for betweenness centrality. *ACM SIGPLAN Notices* 51, 8 (2016), 1–13.